# Presentación




**Nombre:** Jomil José Mota

**Matricula:** 100617004

**Asignatura:** Lab Lenguaje de Program III

**Sección:** 01

# Introducción

El desarrollo de software moderno requiere una estructura organizada que facilite el mantenimiento, la escalabilidad y la reutilización del código. El patrón MVC (Modelo-Vista-Controlador) es una de las arquitecturas más utilizadas para separar las responsabilidades dentro de una aplicación, permitiendo una mejor organización entre la lógica de negocio, la interfaz de usuario y el control de las interacciones. En este trabajo se estudiará el origen de MVC, sus componentes principales, ventajas y desventajas, su importancia en la programación actual, su implementación en frameworks modernos y un ejemplo práctico que permita comprender su funcionamiento.

1.- ¿Qué es el patrón MVC?

Modelo-vista-controlador (MVC) es un patrón de arquitectura de software, que separa los datos y principalmente lo que es la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones. Para ello MVC propone la construcción de tres componentes distintos que son el modelo, la vista y el controlador; es decir: por un lado define componentes para la representación de la información y, por otro lado, para la interacción del usuario. Este patrón de arquitectura de software se basa en las ideas de reutilización de código y la separación de conceptos, características que buscan facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento.

2.- ¿Cuál es el origen del modelo MVC y para qué fue creado?

El patrón MVC fue una de las primeras ideas en el campo de las interfaces gráficas de usuario y uno de los primeros trabajos en describir e implementar aplicaciones software en términos de sus diferentes funciones.

MVC fue introducido por Trygve Reenskaug (web personal) en Smalltalk-76 durante su visita a Xerox Parc en los años 70, seguidamente, en los años 80, Jim Althoff y otros implementaron una versión de MVC para la biblioteca de clases de Smalltalk-80. Solo más tarde, en 1988, MVC se expresó como un concepto general en un artículo sobre Smalltalk-80.

En esta primera definición de MVC el controlador se definía como «el módulo que se ocupa de la entrada» (de forma similar a como la vista «se ocupa de la salida»). Esta definición no tiene cabida en las aplicaciones modernas en las que esta funcionalidad es asumida por una combinación de la 'vista' y algún framework moderno para desarrollo. El 'controlador', en las aplicaciones modernas de la década de 2000, es un módulo o una sección intermedia de código, que hace de intermediario de la comunicación entre el 'modelo' y la 'vista', y unifica la validación (utilizando llamadas directas o el «observer» para desacoplar el 'modelo' de la 'vista' en el 'modelo' activo[).

3.- Explicación de cada componente del modelo: Modelo, Vista y Controlador.

El Modelo: Es la representación de la información con la cual el sistema opera, por lo tanto gestiona todos los accesos a dicha información, tanto consultas como actualizaciones, implementando también los privilegios de acceso que se hayan descrito en las especificaciones de la aplicación (lógica de negocio). Envía a la 'vista' aquella parte de la información que en cada momento se le solicita para que sea mostrada (típicamente a un usuario). Las peticiones de acceso o manipulación de información llegan al 'modelo' a través del 'controlador'. El Controlador: Responde a eventos (usualmente acciones del usuario) e invoca peticiones al 'modelo' cuando se hace alguna solicitud sobre la información (por ejemplo, editar

un documento o un registro en una base de datos). También puede enviar comandos a su 'vista' asociada si se solicita un cambio en la forma en que se presenta el 'modelo' (por ejemplo, desplazamiento o scroll por un documento o por los diferentes registros de una base de datos), por tanto se podría decir que el 'controlador' hace de intermediario entre la 'vista' y el 'modelo' (véase Middleware). La Vista: Presenta el 'modelo' (información y lógica de negocio) en un formato adecuado para interactuar (usualmente la interfaz de usuario), por tanto requiere de dicho 'modelo' la información que debe representar como salida.

4.- Ventajas y desventajas del uso de MVC. Ventajas

Separación de responsabilidades: Cada parte del sistema cumple una función específica. El modelo se centra en los datos y la lógica del negocio, la vista en la presentación, y el controlador en la comunicación entre ambas. Esta división evita la mezcla de tareas y simplifica el mantenimiento del proyecto.

Mayor facilidad para mantener y escalar: Cuando los componentes están bien delimitados, modificar o añadir nuevas funcionalidades resulta mucho más sencillo. ¿Por qué? Porque el desarrollador encargado de dichas modificaciones será capaz de actualizar la vista sin alterar la lógica del modelo, o adaptar la estructura sin afectar la experiencia del usuario.

Reutilización del código: Las reglas de negocio definidas en el modelo pueden aprovecharse en diferentes partes de la aplicación. Esto reduce la duplicación de código, mejora la consistencia y acelera el desarrollo de nuevas funciones.

Desventajas

Complejidad inicial: Al implementar MVC, puede haber una curva de aprendizaje para entender cómo funciona cada componente y cómo interactúan entre sí. Esto puede resultar en una complejidad inicial para los desarrolladores nuevos en este patrón arquitectónico. Duplicación de código: En algunos casos, MVC puede llevar a la duplicación de código, especialmente en proyectos pequeños donde la separación estricta de responsabilidades puede resultar en fragmentación del código y dificultades para mantenerlo. Rendimiento: La separación de responsabilidades en MVC puede llevar a un mayor número de solicitudes entre el modelo, la vista y el controlador, lo que podría afectar al rendimiento de la aplicación. Es importante optimizar adecuadamente cada componente para evitar cuellos de botella. Sobrecarga: En proyectos pequeños o simples, la implementación completa de MVC puede resultar en una sobrecarga innecesaria. A veces, la simplicidad es preferible a la estructura excesiva que ofrece MVC. Flexibilidad limitada: Aunque MVC proporciona una clara separación de preocupaciones, esta misma estructura puede limitar la flexibilidad cuando se requieren cambios significativos en la aplicación. Modificar una capa del modelo podría afectar a las otras capas y requerir ajustes adicionales.

5.- ¿Por qué MVC sigue siendo importante en la programación actual?

El patrón de diseño Modelo-Vista-Controlador (MVC) ha sido un pilar en el desarrollo de software desde su creación en la década de los 70. Aunque han surgido nuevas arquitecturas como MVVM, VIPER e incluso arquitecturas serverless diseñadas para resolver problemas modernos, MVC sigue siendo uno de los patrones más utilizados en el desarrollo de aplicaciones tanto web como móviles.

La separación de responsabilidades no solo hace que el código sea más fácil de entender y mantener, sino que también promueve la reutilización y escalabilidad, elementos cruciales para proyectos modernos. En un mundo donde la agilidad y la eficiencia son clave, la simplicidad de MVC sigue siendo su mayor fortaleza.

6.- ¿Cómo se usa MVC hoy en día en frameworks modernos?

Los primeros frameworks MVC para desarrollo web planteaban un enfoque de cliente ligero en el que casi todas las funciones, tanto de la vista, el modelo y el controlador recaían en el servidor. En este enfoque, el cliente manda la petición de cualquier hiperenlace o formulario al controlador y después recibe de la vista una página completa y actualizada (u otro documento); tanto el modelo como el controlador (y buena parte de la vista) están completamente alojados en el servidor. Como las tecnologías web han madurado, ahora existen frameworks como JavaScriptMVC, Backbone o jQuery que permiten que ciertos componentes MVC se ejecuten parcial o totalmente en el cliente.

7.- Comparar el MVC tradicional con la forma en que se aplica actualmente en tecnologías como Spring MVC, ASP.NET MVC, Laravel, Django, Ruby on Rails u otros frameworks similares.

El MVC tradicional se diseñó para interfaces de escritorio, donde la Vista y el Controlador coexisten en el mismo proceso y comparten estado instantáneamente. En contraste, los frameworks web modernos (Laravel, Django, Ruby on Rails, Spring MVC, ASP.NET Core) adaptan este patrón al entorno HTTP stateless, utilizando un Front Controller como punto de entrada único y serializando datos en lugar de compartir estado en memoria.

Aunque la filosofía central de separar Modelos (datos), Vistas (interfaz) y Controladores (lógica de petición) se mantiene, existen diferencias estructurales clave:

Nomenclatura y Flujo: En Django (Python), el patrón se denomina MTV (Model-Template-View), donde la "Vista" actúa como el Controlador y la "Plantilla" como la Vista. En Rails (Ruby), se aplica el principio de "convención sobre configuración", automatizando la generación de estas capas. Acoplamiento Frontend/Backend: El MVC web tradicional acopla la lógica de negocio con el renderizado de la interfaz en

el servidor. La arquitectura moderna tiende hacia APIs RESTful, donde el MVC maneja la lógica backend y serialización (JSON), dejando el renderizado de la interfaz a frameworks frontend independientes (React, Angular) que suelen usar patrones como MVVM. Complejidad y Capas: Frameworks empresariales como Spring MVC (Java) o ASP.NET Core (C#) suelen añadir capas adicionales (Servicios, Repositorios) sobre el MVC básico para manejar inyección de dependencias, transacciones y pruebas unitarias, lo que lo hace más verboso pero robusto para sistemas de alta concurrencia.

8.- Incluir un ejemplo sencillo donde se identifique claramente el Modelo, la Vista y el Controlador.

Sistema de Registro de Biblioteca

Modelo

El modelo representa y gestiona los datos de los libros almacenados en el sistema. Contiene información como código del libro, título, autor, año de publicación y cantidad disponible. Además, se encarga de acceder y actualizar estos datos en la base de datos.

Vista

La vista es la interfaz que utiliza el bibliotecario para interactuar con el sistema. Consiste en formularios y pantallas que permiten registrar nuevos libros, consultar el catálogo y visualizar los resultados de las búsquedas.

Controlador

El controlador recibe las acciones realizadas por el usuario desde la vista, valida la información ingresada y se comunica con el modelo para guardar o consultar los datos. Luego devuelve una respuesta a la vista para mostrar el resultado correspondiente.

9.- Conclusión personal sobre la importancia de comprender MVC antes de trabajar con frameworks modernos. Comprender Modelo-Vista-Controlador (MVC) es esencial antes de usar frameworks modernos porque esta arquitectura es la base estructural de la mayoría de los marcos de desarrollo actuales, como Laravel, Django, Ruby on Rails y ASP.NET Core.

Dominio de la arquitectura: El patrón MVC separa la lógica de negocio (Modelo), la interfaz de usuario (Vista) y el flujo de control (Controlador); entender esta división permite aplicar correctamente las convenciones del framework en lugar de solo memorizar sintaxis.

Escalabilidad y mantenimiento: Conocer las responsabilidades de cada capa facilita la creación de aplicaciones modulares, mantenibles y escalables, evitando la sobreingeniería o el uso incorrecto de componentes.

Eficiencia en el equipo: La separación clara de responsabilidades que promueve MVC permite que los equipos de frontend y backend trabajen de forma simultánea e independiente, acelerando el desarrollo profesional.

# Conclusión

El patrón MVC ha demostrado ser una solución efectiva para desarrollar aplicaciones organizadas, mantenibles y escalables. Aunque los frameworks modernos han adaptado y extendido su implementación original, los principios fundamentales de separación de responsabilidades continúan siendo esenciales en el desarrollo de software. Comprender cómo interactúan el Modelo, la Vista y el Controlador permite aprovechar mejor herramientas como Spring MVC, ASP.NET MVC, Laravel, Django y Ruby on Rails. Por ello, conocer MVC constituye una base sólida para cualquier desarrollador que desee trabajar de manera profesional con las tecnologías y frameworks actuales.

# Bibliografía

https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador

https://www.ebiseducation.com/mvc-que-es-caracteristicas-y-ejemplos

https://www.byronvargas.com/web/que-desventajas-tiene-la-arquitectura-o-modelo-mvc/

https://robles.guru/articulos/mvc-el-patron-de-desarrollo-que-sigue-vigente-en-2023